

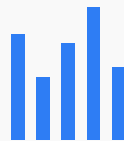
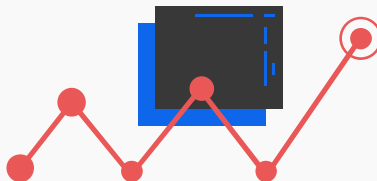
Using React JS

Let's start...





JavaScript



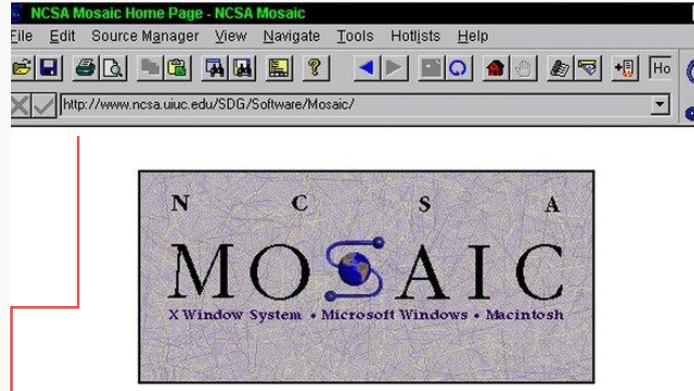
(({ > 0 | □ □ □ }))



➤ What is JavaScript

Stackoverflow Survey

➤ What is JavaScript



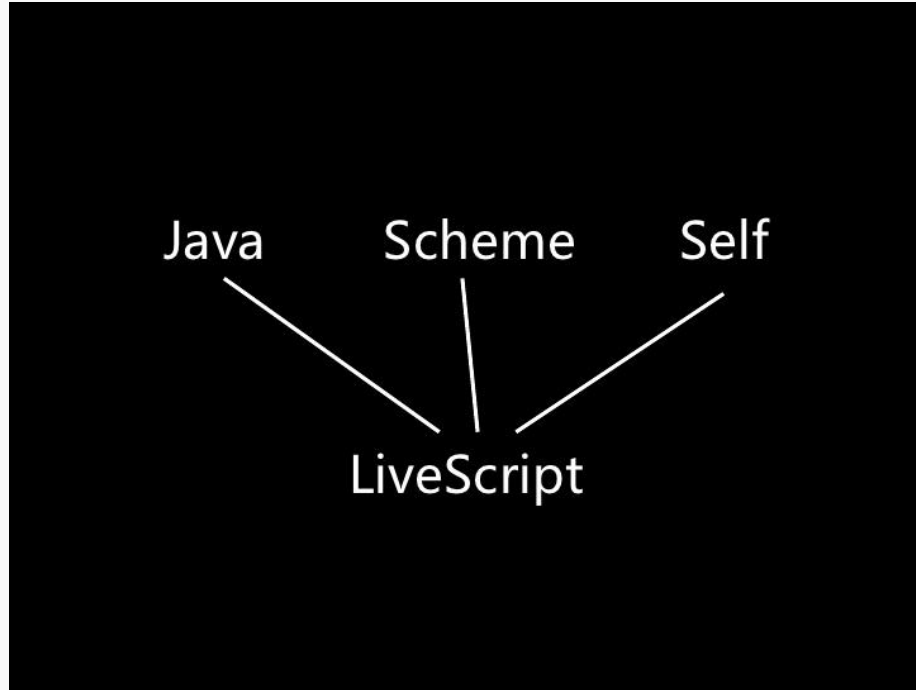
> What is JavaScript

Brendan Eich



JavaScript was created by Brendan Eich in 10 days
while working at Netscape in 1995

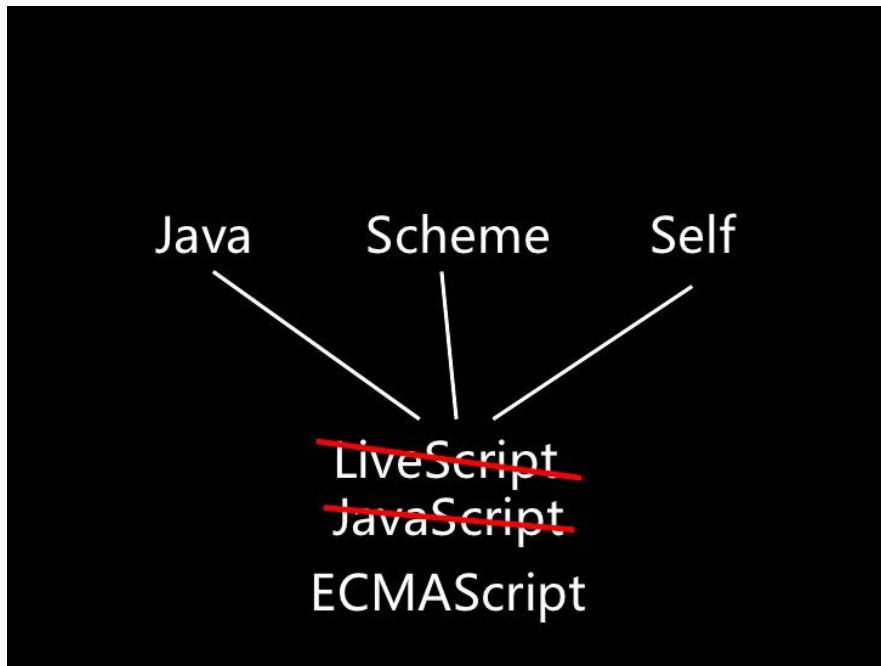
> History of JavaScript



> History of JavaScript



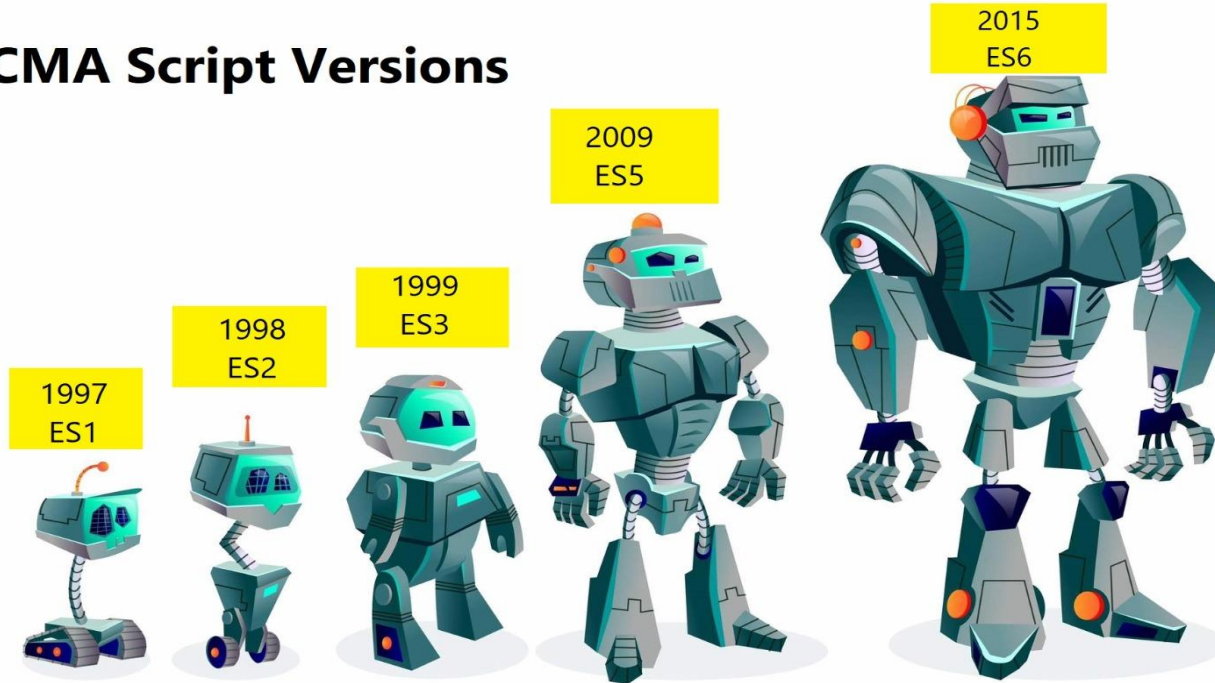
> History of JavaScript



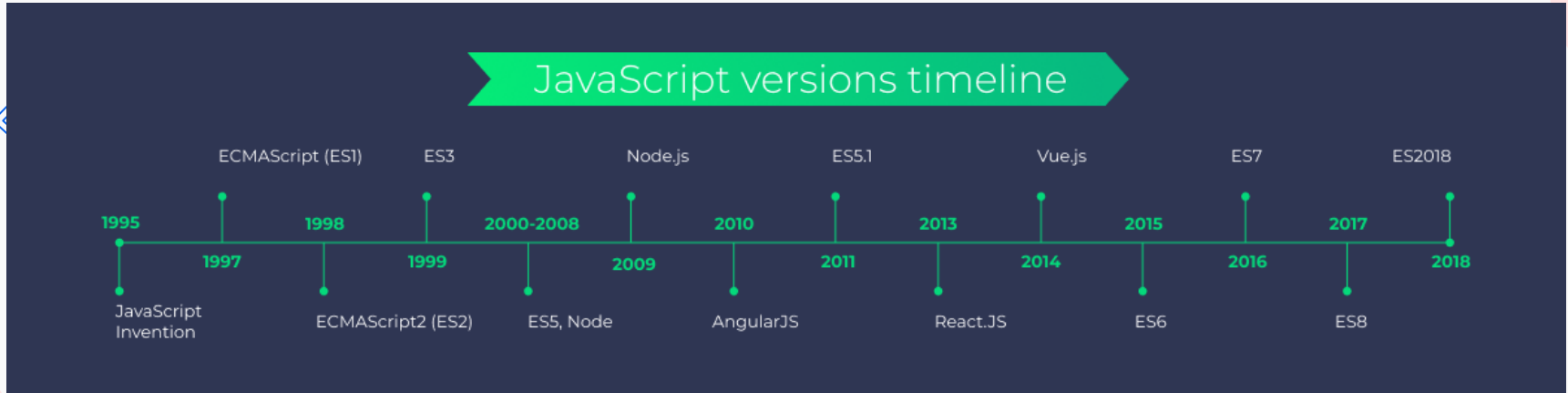
ECMAScript, established in 1997, laid the foundation
for JavaScript's standardization

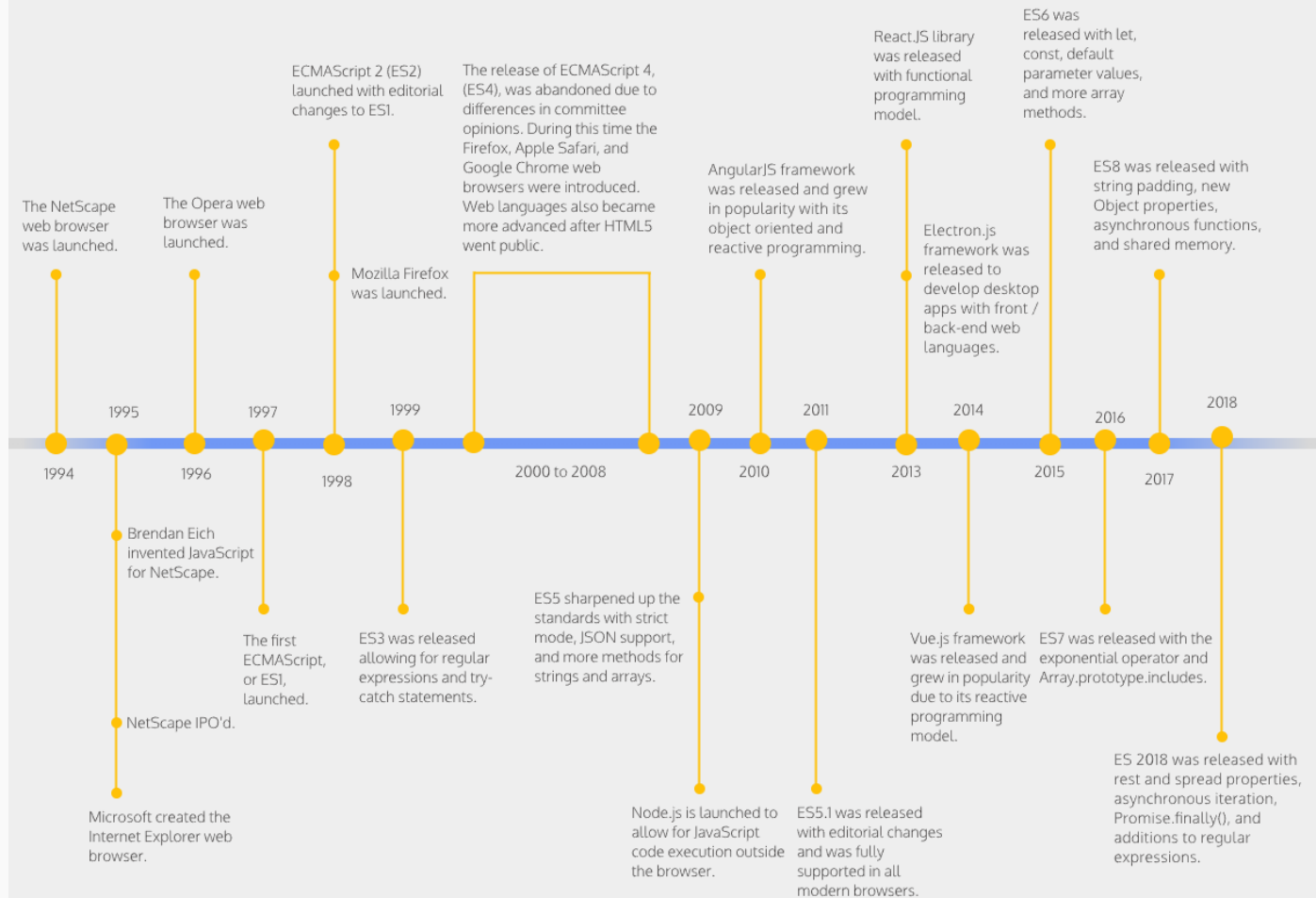
> History of JavaScript

ECMA Script Versions

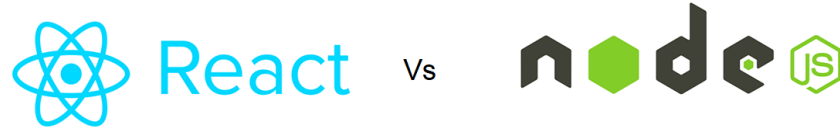


> History of JavaScript





> History of JavaScript



- JavaScript evolved beyond the browser with the advent of Node.js, allowing server-side development.
- Frameworks like React, Angular, and Vue.js further propelled JavaScript's popularity for building dynamic web applications.

➤ Why JavaScript?

[] Why JavaScript is essential for web development?

- 
- JavaScript is supported by all major browsers, making it a universal scripting language for client-side scripting. Compatibility ensures a consistent user experience across different platforms and devices.
 - JavaScript enables dynamic and interactive web pages, enhancing user experience.
 - With the rise of Node.js, JavaScript can be used for both client-side and server-side development.
 - This allows developers to build end-to-end applications using a single language, simplifying the development process.
- 

› Let's play with JavaScript!

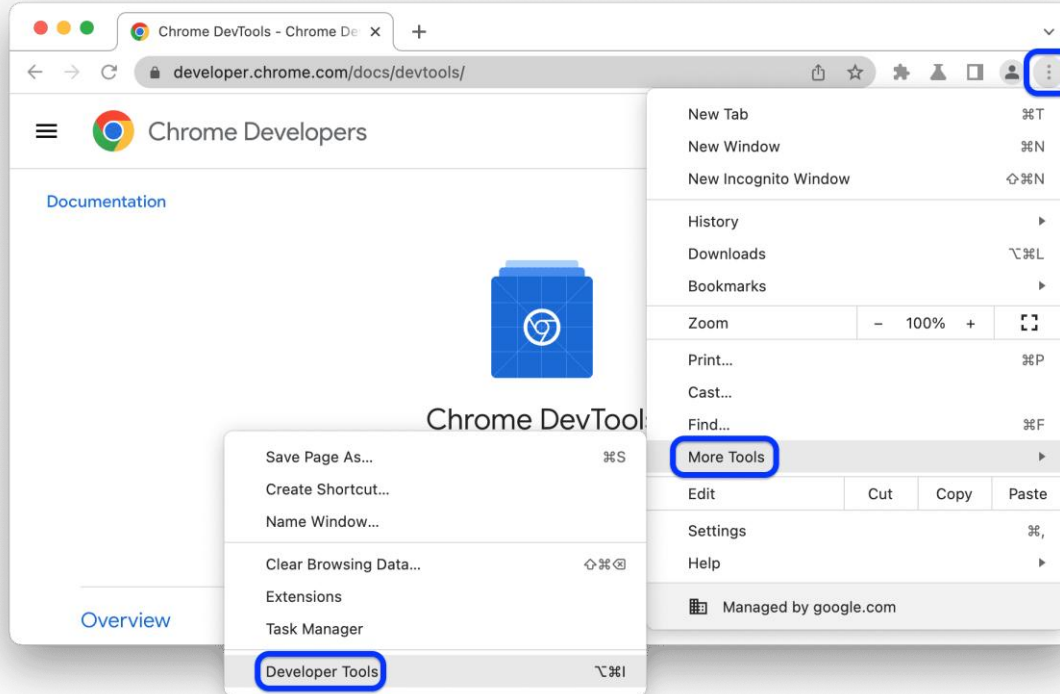
Let's Play

- ITI Website
- `document.getElementsByClassName("section-title__title")[0].style.color = "#901b20";`
- `document.body.addEventListener('click', function () { const myParent = document.getElementsByClassName("centered-section-title__title")[0]; const myImage = document.createElement("img"); myImage.src = 'https://thecatapi.com/api/images/get?format=src&type=gif'; myParent.appendChild(myImage); myImage.style.marginLeft = "160px"; });`
- Now click on body >

› Where to write JavaScript!

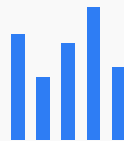
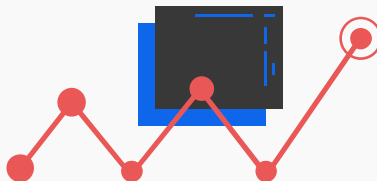


> JavaScript with Dev Tools!





JS Basics



`(( { > 0 | □ □ □ } ))`



> Hello World!

- `Window.alert()`
- `Console.log()`
- `Document`

JS JavaScript PROGRAMS

HELLO WORLD PROGRAM

```
1 // file.js~
2 alert('Hello, World!');~
3 ~
```

> Console.log

The console.log method

- console.log is a fundamental method for outputting information to the console.
- It is commonly used for debugging purposes and displaying variable values during development.
- You can log multiple values in a single console.log statement by separating them with commas.
- Use string interpolation or formatting to enhance log readability. This is particularly useful when logging complex objects or combining strings and variables.

ITI - MOSTAFA ELALFY

```
let variable = "Hello, World!";
console.log(variable);
window.alert(variable);
```

ITI - MOSTAFA ELALFY

```
let user = { name: "Alice", age: 30 };
console.log(`User: ${user.name}, Age:
${user.age}`);
```

ITI - MOSTAFA ELALFY

```
for(I = 0; I < 10; i++) {
  console.log(I);
}
```

➤ JavaScript Comments

```
/**
 * * This is a important comment
 * ! This is warning comment
 * ? This is a question comment
 * TODO: This is a todo comment
 */
```

> JavaScript Case Sensitive

[]

<<

[]

{ }

Case-sensitive



eXAMPLEpassword

≠



examplepassword

➤ JavaScript Semicolon

[]

<<

{ }

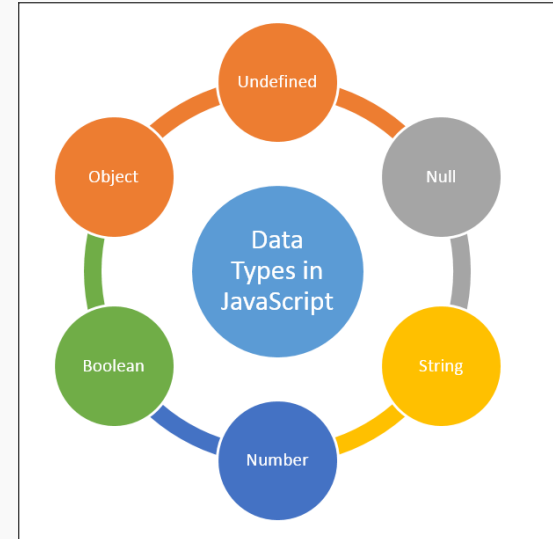
[]

;

> JS Data Types

Data Types

- Primitive data type & Non Primitive data type
- String
- Number
- Boolean
- Undefined
- Null



> JS Data Types

[]

<<

{ }

[]

Non-zero value



null



0



undefined



> JS Data Types

[]

Numbers

Numbers in JavaScript can be integers or floating-point values. They can be declared directly or through mathematical operations.



Mathematical Operations: JavaScript supports standard mathematical operations such as addition, subtraction, multiplication, division, Modulo and Power. The Math object provides additional mathematical functions.

Type Conversion: JavaScript allows for automatic and explicit type conversion. Use functions like `parseInt` or `parseFloat` for explicit conversion between strings and numbers.

{ }

ITI - MOSTAFA ELALFY

```
var integerNumber = 42;
var floatNumber = 3.14;
var result = 10 + 20;
// result is a number
```

ITI - MOSTAFA ELALFY

```
var sum = 10 + 5;
var product = 7 * 3;
var squareRoot = Math.sqrt(25);
```

ITI - MOSTAFA ELALFY

```
var numberAsString = "42";
var convertedNumber = parseInt(numberAsString);
```

> JS Data Types

[]

Strings

Strings in JavaScript can be declared using single or double quotes.

- Template literals (introduced in ES6) allow for multi-line strings and string interpolation.
- **String Methods:** JS provides a set of predefined methods for strings like **length**, **toUpperCase**, **toLowerCase**, **substring**, and **indexOf**.
- **String Concatenation:** Strings can be concatenated using the + operator or template literals. Template literals provide a more readable and flexible way to combine strings and variables.

<<

```
ITI - MOSTAFA ELALFY

var singleQuotes = 'Single Quotes';
var doubleQuotes = "Double Quotes";
var templateLiteral = `Template Literal`;
console.log(typeof singleQuotes);
```

```
ITI - MOSTAFA ELALFY

var name = "John";
var greeting = f"Hello, {name}!";
```

```
ITI - MOSTAFA ELALFY

var firstName = "John";
var lastName = "Doe";
var fullName = firstName + " " + lastName;
// or
var fullNameTemplate = `${firstName} ${lastName}`;
```

```
ITI - MOSTAFA ELALFY

var text = "JavaScript is amazing!";
console.log(text.length);
console.log(text.toUpperCase());
console.log(text.indexOf("amazing"));
```

[]

> JS Arithmetic Operators

Arithmetic Operator	Name	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus	$a \% b$
++	Increment Operator	$a++$
--	Decrement Operator	$a--$

> JS Comparison Operators

Operators	Meaning	Example	Result
<	Less than	5<2	False
>	Greater than	5>2	True
<=	Less than or equal to	5<=2	False
>=	Greater than or equal to	5>=2	True
==	Equal to	5==2	False
!=	Not equal to	5!=2	True
===	Equal value and same type	5 === 5	True
		5 === "5"	False
!==	Not Equal value or Not same type	5 !== 5	False
		5 !== "5"	True

> JS Logical Operators

[]

<<

{ }

[]

&& (AND)	true	&&	true	→	true
	true	&&	false	→	false
	false	&&	false	→	false
 (OR)	true		true	→	true
	true		false	→	true
	false		false	→	false
! (NOT)		!	true	→	false
		!	false	→	true

➤ JS Indexing & Escaping

[]

Strings, Indexing and Escaping

«

- Strings represent text and are enclosed in single or double quotes.
- Indexes and Escaping
- Access characters in a string using indexes.
- Escape characters with a backslash (\) for special characters.
- Strings can be concatenated using the + operator.

{ }

[]

> JS Indexing & Escaping

[]

Strings, Indexing and Escaping

- "Web"[2];
- "The man whispered, "please speak to me.""
"The man whispered, \"please speak to me.\""
- "Up up\n\tdown down"

<<

```
const my_string = "a";
const ASCII_value = my_string.charCodeAt(0);
console.log(ASCII_value);
```

{ }

Special characters

- \\
- "
- '
- \n
- \t

[]

- const my_string = "Web";
for (let i = 0; i < my_string.length; i++) {
- console.log(my_string.charCodeAt(i));
- }

➤ JS Control The Flow

[]

Control flow in JavaScript

- Control flow refers to the order in which statements are executed in a script.
- In JavaScript, control flow is achieved through conditional statements and loops.
- It helps in making decisions and executing different code blocks based on certain conditions.

<<

{ }

[]

```

ITI - MOSTAFA ELALFY

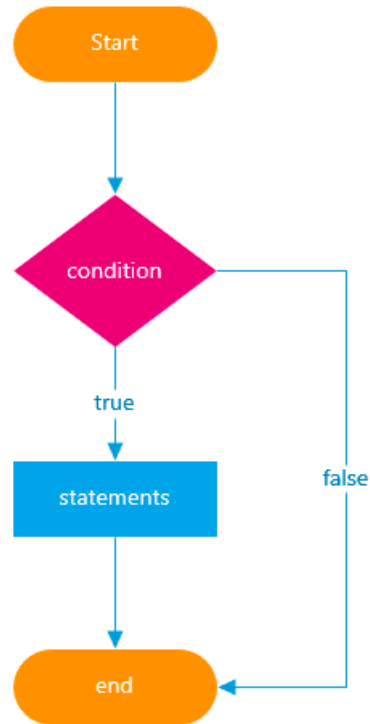
var condition = true;

if (condition) {
  console.log("Condition is true");
} else {
  console.log("Condition is false");
}
  
```


> JS Control The Flow

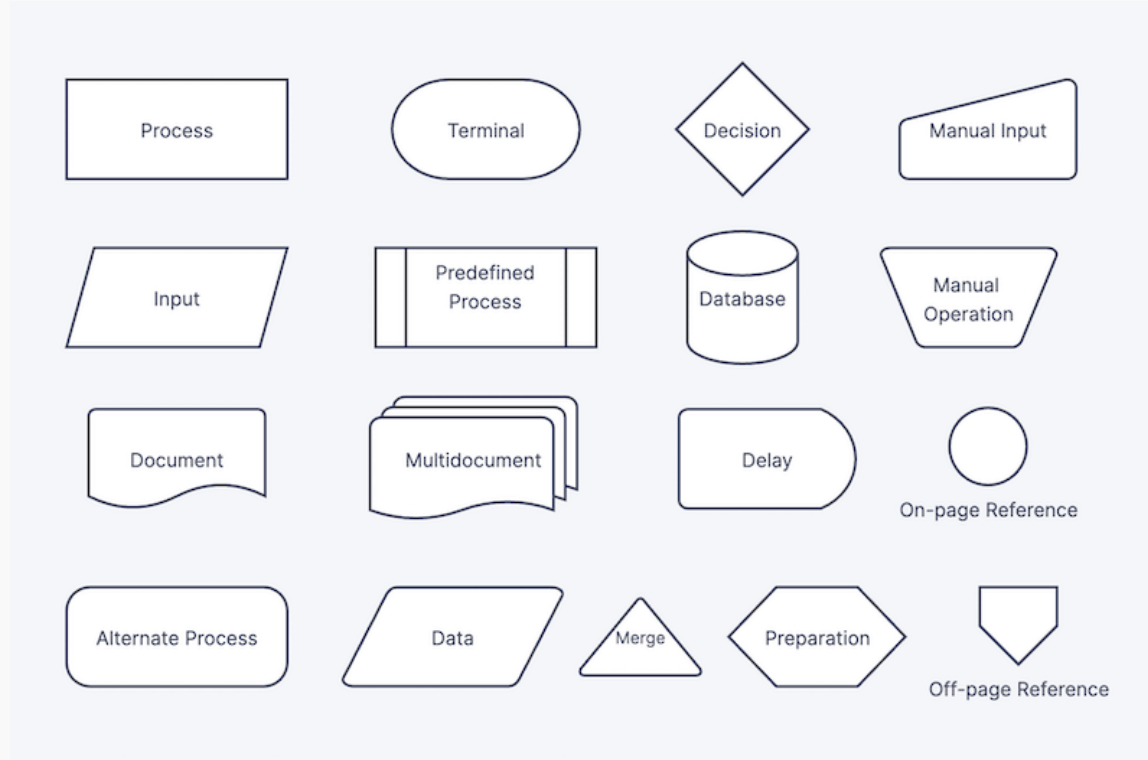
Converting Flowcharts to Code

- Flowcharts are visual representations of a process, and coding involves translating these processes into executable code.
- Analyzing the flow of a program helps in writing structured and logical code.
- Translating a flowchart into code involves using control flow statements to implement the depicted decision-making and looping processes.



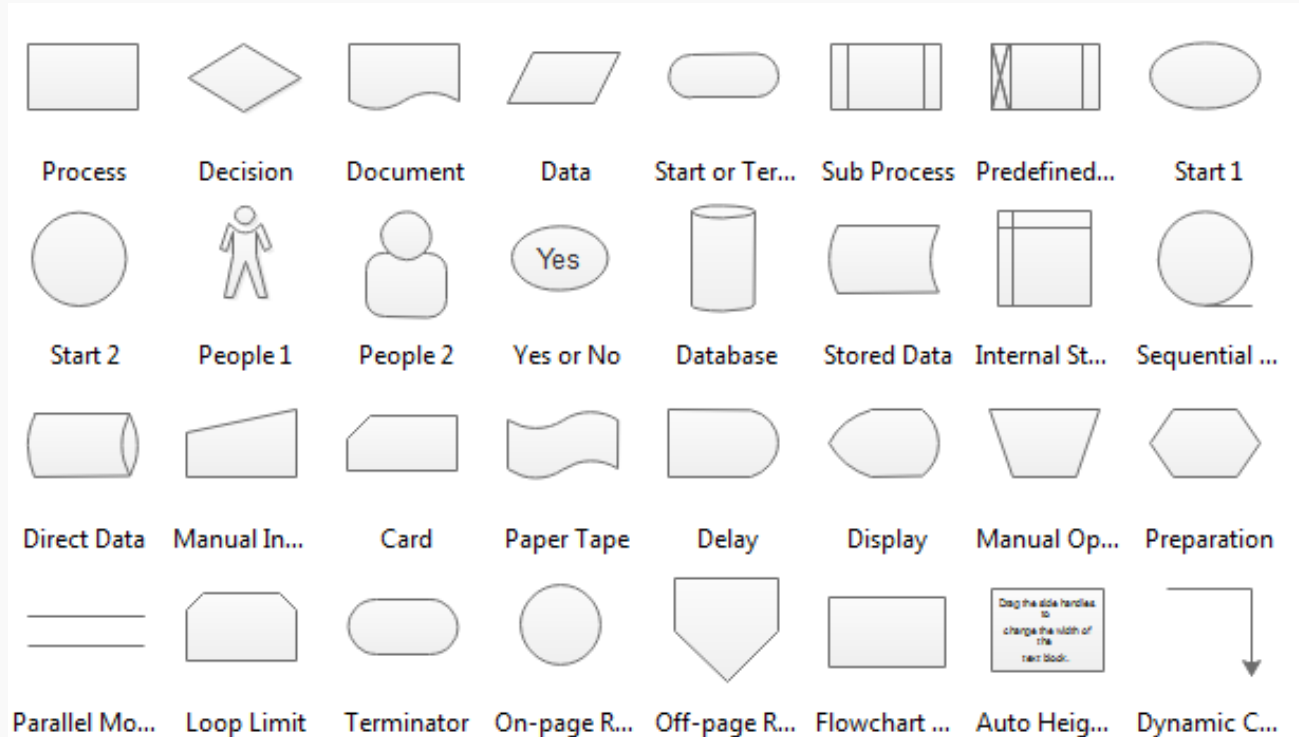
> JS Control The Flow

Flowcharts shapes



> JS Control The Flow

Flowcharts shapes



> JS Control The Flow

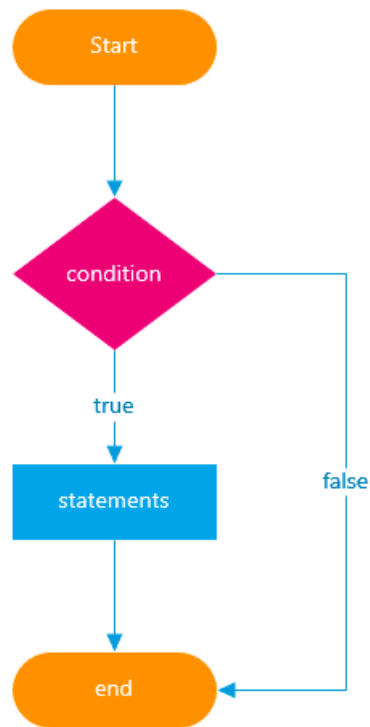
[] Converting Flowcharts to Code

ITI - MOSTAFA ELALFY

```
// Example flowchart:
// Start -> Check condition -> If true, do
// something -> If false, do something else -> End

var condition = true;

if (condition) {
  console.log("Do something");
} else {
  console.log("Do something else");
}
```



> JS IF Statement

IF Syntax

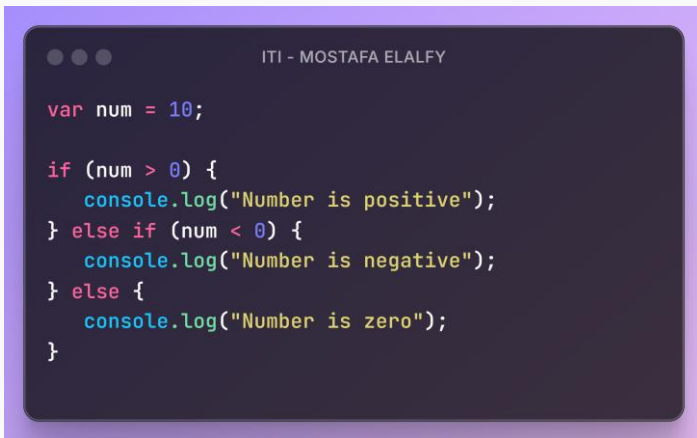
- The if statement in JavaScript allows for basic decisions based on conditions.

```
ITI - MOSTAFA ELALFY  
  
var num = 10;  
  
if (num > 0) {  
  console.log("Number is positive");  
}
```

> JS IF Else Statement

If Else Syntax

- The if-else statement in JavaScript allows for branching based on conditions.
- The else if statement extends the decision-making process to handle multiple conditions.
- It provides a way to create a series of conditions and execute the corresponding block of code for the first true condition encountered.



```
ITI - MOSTAFA ELALFY

var num = 10;

if (num > 0) {
  console.log("Number is positive");
} else if (num < 0) {
  console.log("Number is negative");
} else {
  console.log("Number is zero");
}
```

➤ JS Nested IF Statement

[]

Nested IF Syntax

<<

{ }

[]

```
ITI - MOSTAFA ELALFY

var age = 20;
var hasLicense = true;

if (age >= 18) {
  if (hasLicense) {
    console.log("You can drive.");
  } else {
    console.log("You need a license to drive.");
  }
} else {
  console.log("You are too young to drive.");
}
```

> JS Switch Statement

[]

Switch Statement

<<

- The switch statement is an alternative to multiple if-else if statements.
- It evaluates an expression against multiple possible case values and executes the code block associated with the first matching case.
- It provides a cleaner structure when dealing with multiple conditions.

{ }

[]

```
ITI - MOSTAFA ELALFY

var dayOfWeek = "Monday";

switch (dayOfWeek) {
  case "Monday":
    console.log("It's the start of the week");
    break;
  case "Friday":
    console.log("Weekend is approaching");
    break;
  default:
    console.log("Some other day");
}
```


> JS Ternary Operator

Ternary Operator

- The ternary operator is a shorthand way of writing an if-else statement in a single line.
- It's concise and useful when assigning values based on a condition.
- Syntax: condition ? expression_if_true : expression_if_false

```

ITI - MOSTAFA ELALFY

var condition = true;

var result = condition ? "Value if true" : "Value if false";
console.log(result);
  
```

Thanks!

I hope you enjoyed the session.



{({({ >> })) } << }

